

Tyler Vander Mooren

ITC 413: NoSQL Databases

Professor Bryan Knueppel

March 17, 2026

Assignment 9

Part 1: Analysis Phase

Metro City's current relational database is organized in a logical and normalized way, but it is not well suited for real-time traffic workloads. The schema separates major entities such as intersections, traffic sensors, traffic incidents and others into related tables connected through foreign keys. This design supports consistency and integrity, but it becomes inefficient when the system starts processing more than 50,000 sensor readings per minute while also serving live dashboards and analytics. The largest bottleneck is the sensor readings portion of the schema because it is receiving continuous, high-volume writes while also serving as the base for operations and analytics. In a relational design, answering the questions about current traffic conditions often requires joining sensor readings with traffic sensors and intersections to be able to identify location and context. The more advanced operational views also require joins with incidents and signals. While under heavy writing loads, these repeated joins increase the latency and reduce overall performance. This becomes especially problematic because the same system is expected to support real-time ingestion and complex analysis at the same time.

Traditional relational approaches struggle in this scenario for three main reasons. Indexing a rapidly growing table to support many query patterns will increase the write overhead. Additionally, joins across very large tables raise latency for time-sensitive operational queries. Also, historical analysis over large-time windows requires scanning and aggregating

significant amounts of data before producing any useful results. Column family systems are better aligned to deal with this type of use case because they are designed around known query patterns, distributed storage and large-scale time series workloads (Wu et al., 2018; Sullivan, 2015). These limitations affect all the stakeholder groups. The Traffic Control Center needs district wide real time conditions, but with normalized joins it can delay dashboard refreshes. Emergency Services require immediate traffic awareness, yet relational queries may not return current congestion or incident related data quickly enough during urgent situations. City Planners need long term trend analysis by the hour, day and month but having repeated scans over large readings is expensive. Public Information systems also depend on low latency reads for their mobile applications and websites, so citizens may experience slower traffic updates when the relational database is under heavy load.

For this redesign, the three most important query patterns are sensor data ingestion, real time traffic dashboard queries and historical analytics. Sensor ingestion is the core because every other function depends on accurate and timely incoming data. The real time dashboard is also critical because it supports immediate traffic management and public information. Historical analytics is also equally important because the city must use past traffic trends to improve infrastructure planning and to optimize signal timing. In the current relational model, these patterns all depend heavily on normalized tables and joins, which limits performance at scale.

Part 2: Column Family Design

A column family design should first start with the city's most important queries rather than with normalization rules. Column family databases perform best when the tables are designed specifically for the reads and writes the application must execute most often. This

follows the principle of designing for queries rather than designing for general flexibility (Wu et al., 2018; Sullivan, 2015).

Keyspace Definition

```
CREATE KEYSPACE metro_traffic
WITH replication = {
  'class': 'NetworkTopologyStrategy',
  'metro_dc1': 3
};
```

Using the NetworkTopologyStrategy replication strategy was chosen because Metro City's traffic platform is operationally critical and should remain available even if a single node fails. A replication factor of 3 provides a practical balance between fault tolerance, availability and storage overhead. This replication choice aligns with recommended Cassandra design practices for resilient production environments (Wu et al., 2018).

Core Column Families

1. Intersections by District

```
CREATE TABLE metro_traffic.intersections_by_district (
  district text,
  intersection_id text,
  intersection_name text,
  main_street text,
  cross_street text,
  latitude decimal,
  longitude decimal,
  traffic_light_count int,
  has_pedestrian_crossing boolean,
  priority_level text,
  PRIMARY KEY ((district), intersection_id)
);
```

2. Current Conditions by District

```
CREATE TABLE metro_traffic.current_conditions_by_district (
  district text,
  intersection_id text,
  last_update timestamp,
  intersection_name text,
```

```

avg_speed_mph decimal,
vehicle_count int,
occupancy_percentage decimal,
active_incident boolean,
incident_severity text,
estimated_delay_minutes int,
signal_phase text,
PRIMARY KEY ((district), intersection_id)
);

```

3. Sensor Readings by Intersection Day

```

CREATE TABLE metro_traffic.sensor_readings_by_intersection_day (
  intersection_id text,
  reading_date date,
  event_time timestamp,
  sensor_id text,
  lane_number int,
  vehicle_count int,
  avg_speed_mph decimal,
  occupancy_percentage decimal,
  vehicle_type text,
  weather_condition text,
  visibility_meters int,
  PRIMARY KEY ((intersection_id, reading_date), event_time, sensor_id)
) WITH CLUSTERING ORDER BY (event_time DESC, sensor_id ASC);

```

4. Hourly Traffic Summary by Intersection

```

CREATE TABLE metro_traffic.hourly_traffic_summary_by_intersection (
  intersection_id text,
  summary_date date,
  hour_bucket int,
  avg_speed_mph decimal,
  total_vehicle_count bigint,
  avg_occupancy_percentage decimal,
  reading_count int,
  PRIMARY KEY ((intersection_id, summary_date), hour_bucket)
);

```

Row Key Design Strategy

The row keys are designed with a focus on Metro City's highest priority queries. In Cassandra, the partition key determines where the data is stored, while clustering columns can

determine how rows are ordered within that partition. So, good partition-key design is essential for performance (Wu et al., 2018). For the highest volume table, `sensor_readings_by_intersection_day`, the partition key is `(intersection_id, reading_date)`. This keeps the partitions bounded and makes it efficient to retrieve recent data for one location. The clustering columns `event_time` and `sensor_id` allow for the newest readings to appear first, which is critical for both recent operational lookups and time-based analysis. Hotspotting is also reduced by avoiding oversized partitions such as all the readings for a single intersection across all time. Instead, readings are now bucketed by days so the writes are distributed across many partitions over time. This makes the design more scalable and keeps partitions at manageable sizes (Sullivan, 2015).

Part 3: Design Justification Report

Denormalization Strategy

The main denormalization strategy in this design is storing the data required for each major query pattern. In the original relational schema, real-time dashboard queries would require joins across intersections, traffic sensors and sensor readings before returning any useful traffic view. In the column family design, `current_conditions_by_district` stores the latest traffic state for each intersection in a district so the system can read directly from one table instead of performing several joins. Historical analysis is handled the same way. Rather than having to repeatedly scan raw sensor readings, the design uses `hourly_traffic_summary_by_intersection` to store the precomputed hourly summaries. This makes the historical and predictive analysis much faster. These denormalization decisions will eliminate joins for the three selected query patterns because ingestion is now written to `sensor_readings_by_intersection_day`, dashboard queries

read from `current_conditions_by_district` and the analytics queries read from `hourly_traffic_summary_by_intersection` (Sullivan, 2015).

Secondary Index Implementation

For secondary access patterns, tables-as-indexes were used rather than database-managed secondary indexes. Cassandra performs best when access patterns are known in advance and tables are built specifically around those queries. That is what makes purpose-built tables more predictable and scalable than secondary indexes in a high-write environment. This approach fits Metro City's workload because the city already knows its most important operational queries and needs consistent performance while under heavy ingestion rates (Wu et al., 2018; Sullivan, 2015).

Trade-off Analysis

The main benefit of this design is improved scalability and lower query latency. High-volume sensor writes are better distributed across partitions and the most important reads no longer depend on expensive relational joins. This especially benefits the Traffic Control Center, Public Information systems and City Planners as each group depends on fast access to either current traffic conditions or historical summaries. The main tradeoffs are added redundancy, greater application complexity and less support for ad hoc queries. Some values may appear in more than one table and new access patterns may require new tables. However, even with those costs the column family approach is more suited for Metro City because it is optimized for the high throughput and query driven workload that the current relational model struggles to support.

References

Sullivan, D. (2015). *NoSQL for Mere Mortals*. Addison-Wesley Professional.

Wu, X. (Brian), Kadamby, S., Kandhare, D., & Ploetz, A. (2018). *Seven NoSQL Databases in a Week*. Packt Publishing